

Types of Entities in Entity Framework with Examples

In this article, I am going to discuss the **Types of Entities in the Entity Framework** with Examples. Please read our previous article where we discussed [Entities in Entity Framework](#) with Examples. We are going to work with the same example that we have worked on so far.

Types of Entities in Entity Framework

In Entity Framework, there are two types of are as follows:

1. **POCO Entities**
2. **Dynamic Proxy Entities**

POCO Entities in Entity Framework

POCO stands for Plain Old CLR Objects which can be used as existing domain objects with your data model. The POCO data classes which are mapped to entities are defined in a data model. A POCO entity is a class that does not depend on any framework-specific base class. It is like any other normal .NET CLR class, which is why it is called “Plain Old CLR Objects”.

These POCO entities support most of the same query, insert, update, and delete behaviors as entity types that are generated by the Entity Data Model. The following is an example of an Employee POCO entity that is created by the Entity Framework.

```
public class Employee
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
    public string Gender { get; set; }
    public Nullable<int> Salary { get; set; }
    public Nullable<int> DepartmentId { get; set; }
    public virtual Department Department { get; set; }
}
```

Dynamic Proxy Entities

When creating instances of POCO entity types, the Entity Framework often creates instances of a dynamically generated derived type that acts as a proxy for the entity. It can also be said that it is a runtime proxy class like a wrapper class of a POCO entity.

The Dynamic Proxy is a runtime proxy class that wraps the POCO entity. A POCO entity should meet the following requirements to become a POCO Proxy Entity:

1. The POCO class must be declared with Public Access Modifier.
2. A POCO class must not be sealed.
3. The POCO class must not be abstract.
4. Each navigation property must be declared as public, and virtual.
5. All the collection properties must be ICollection<T> type.
6. The ProxyCreationEnabled option must NOT be false (default is true) in the context

The following POCO entity meets all the above requirements to become a dynamic proxy entity at runtime.

```
public class Employee
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
    public string Gender { get; set; }
    public Nullable<int> Salary { get; set; }
    public Nullable<int> DepartmentId { get; set; }
    public virtual Department Department { get; set; }
}
```

Get Entity Type from Proxy Type in C#:

We can find the original or actual entity type from the proxy type using the `GetObjectType` method of the `ObjectContext` class. The `GetObjectType` is a static method so there is no need to create an object of the object `ObjectContext`, we can call that method directly using the class name.

In the below example, the `DBEntities.Employees.Find(1)` method will return the dynamic proxy for Employee Type. Here, the `Find` method will accept one value i.e. the Primary key column value, and will return that Employee data. Here, the .NET Framework will create an instance of the dynamically generated proxy class of Employee Entity.

The `GetObjectType` method returns the actual entity type even if we pass the instance of the entity type (not the proxy type). In short, we can always use this method to get the actual entity type, there is no need to check whether the type is a proxy type or not.

```
using System;
using System.Data.Entity.Core.Objects;
namespace EFDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            using (EF_Demo_DBEntities DBEntities = new EF_Demo_DBEntities())
            {
                //It will return the dynamic proxy for Employee Type
                //Employee Type Dynamic Proxy:
                System.Data.Entity.DynamicProxies.Employee
                var employee = DBEntities.Employees.Find(1);
                Console.WriteLine($"Dynamic Proxy For Employee: {employee}");
                //Use ObjectContext.GetObjectType() to find the underlying
                wrapped type
                //by the Dynamic Proxy
                var employeeType =
                ObjectContext.GetObjectType(employee.GetType());
                Console.WriteLine($"Wrapped Type By Dynamic Proxy:
                {employeeType}");
                Console.WriteLine($"FullName of Employee Master Type :
                {employeeType.FullName}");
            }
        }
    }
}
```

```

        Console.WriteLine($"Name of Employee Master Type :
{employeeType.Name}");
        Console.ReadKey();
    }
}
}
}
}

```

Output:

```

Dynamic Proxy For Employee: System.Data.Entity.DynamicProxies.Employee_BEFF4715893673C3C38294E59
C787F7264739EF7B4B227657A6654A26CB7875B
Wrapped Type By Dynamic Proxy: EFDemo.Employee
FullName of Employee Master Type : EFDemo.Employee
Name of Employee Master Type : Employee

```

Disabling Dynamic Proxy Creation in Entity Framework:

By default, the dynamic proxy is enabled for every entity in the derived context class. However, we can disable the dynamic proxy creation in the context class by setting the `ProxyCreationEnabled` property to false.

- **ProxyCreationEnabled:** It gets or sets a value indicating whether or not the framework will create instances of dynamically generated proxy classes whenever it creates an instance of an entity type. Note that even if proxy creation is enabled with this flag, proxy instances will only be created for entity types that meet the requirements for being proxied. Proxy creation is enabled by default.

Now, to disable dynamic proxy creation, modify the context class which is generated by Entity Framework as follows: Here, you can see, within the constructor we are writing a single line of statement i.e. `this.Configuration.ProxyCreationEnabled = false;` and this statement will now disable the dynamic proxy creation.

```

//-----
---
// <auto-generated>
//     This code was generated from a template.
//
//     Manual changes to this file may cause unexpected behavior in your
// application.
//     Manual changes to this file will be overwritten if the code is
// regenerated.
// </auto-generated>
//-----
---
namespace EFDemo
{
    using System;
    using System.Data.Entity;
    using System.Data.Entity.Infrastructure;

    public partial class EF_Demo_DBEntities : DbContext
    {
        public EF_Demo_DBEntities()

```

```

        : base("name=EF_Demo_DBEntities")
    {
        this.Configuration.ProxyCreationEnabled = false;
    }

    protected override void OnModelCreating(DbModelBuilder modelBuilder)
    {
        throw new UnintentionalCodeFirstException();
    }

    public virtual DbSet<Department> Departments { get; set; }
    public virtual DbSet<Employee> Employees { get; set; }
}
}

```

With the above changes in place, now run the application and you will see the following output. Now, .NET Framework will not create an instance of dynamic proxy.

```

Dynamic Proxy For Employee: EFDemo.Employee
Wrapped Type By Dynamic Proxy: EFDemo.Employee
FullName of Employee Master Type : EFDemo.Employee
Name of Employee Master Type : Employee

```

Now, let us set the ProxyCreationEnabled property to true in the context class as follows:

```

//-----
---
// <auto-generated>
//     This code was generated from a template.
//
//     Manual changes to this file may cause unexpected behavior in your
// application.
//     Manual changes to this file will be overwritten if the code is
// regenerated.
// </auto-generated>
//-----
---
namespace EFDemo
{
    using System;
    using System.Data.Entity;
    using System.Data.Entity.Infrastructure;

    public partial class EF_Demo_DBEntities : DbContext
    {
        public EF_Demo_DBEntities()
            : base("name=EF_Demo_DBEntities")
        {
            //Setting ProxyCreationEnabled to True
            //By Default this property value is also true

```

```

        this.Configuration.ProxyCreationEnabled = true;
    }

    protected override void OnModelCreating(DbModelBuilder modelBuilder)
    {
        throw new UnintentionalCodeFirstException();
    }

    public virtual DbSet<Department> Departments { get; set; }
    public virtual DbSet<Employee> Employees { get; set; }
}
}

```

Next, modify the auto-generated Employee entity class as follows: As you can see, we are removing the virtual keyword from the Department property. Now, the following Employee class is a POCO Entity, but not a Dynamic Proxy Entity as the navigation property Department is not virtual.

```

//-----
---
// <auto-generated>
//     This code was generated from a template.
//
//     Manual changes to this file may cause unexpected behavior in your
// application.
//     Manual changes to this file will be overwritten if the code is
// regenerated.
// </auto-generated>
//-----
---
namespace EFDemo
{
    using System;
    using System.Collections.Generic;

    public partial class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Email { get; set; }
        public string Gender { get; set; }
        public Nullable<int> Salary { get; set; }
        public Nullable<int> DepartmentId { get; set; }

        public Department Department { get; set; }
    }
}

```

As the above Employee Entity is not a Dynamic Proxy Entity, even though the ProxyCreationEnabled is set to true, the .NET Framework is not going to create a dynamic proxy while creating an instance of the Employee entity. Now, run the application with the above changes in place and you will get the following output.

```
Dynamic Proxy For Employee: EFDemo.Employee  
Wrapped Type By Dynamic Proxy: EFDemo.Employee  
FullName of Employee Master Type : EFDemo.Employee  
Name of Employee Master Type : Employee
```

When to use Dynamic Proxy in Entity Framework?

If you want to use lazy loading or change tracking, then you need to use Dynamic Proxy.

1. **Lazy Loading:** Lazy Loading is the default behavior of Entity Framework. That means the related entities or child entities are loaded only when it is being accessed for the first time. That means in simple words we can say that Lazy loading simply delays the loading of related entities until you specifically request it.
2. **Change Tracking:** Entity Framework supports automatic change tracking of the loaded entities during the lifetime of the context. The `DbChangeTracker` class gives you all the information about current entities being tracked by the context.

In our upcoming articles, we will discuss Lazy Loading and Change Tracking in detail.

In the next article, I am going to discuss the [Entity States in Entity Framework](#). Here, in this article, I try to explain the **Types of Entities in the Entity Framework with Examples**. I hope this article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about the Types of Entities in the Entity Framework article.